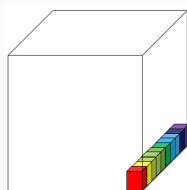
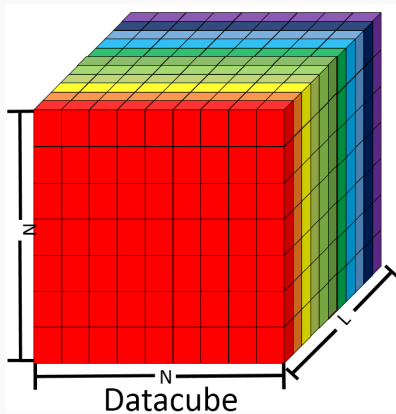


Lecture 4a:

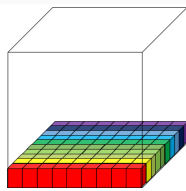
Diffraction Spatio-Spectral Imaging Architecture for Classification Using Deep Learning



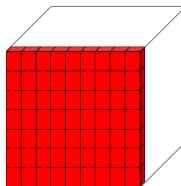
Spectral Imaging



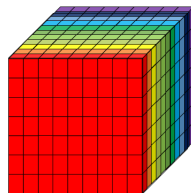
Point scanning



Line scanning



Area scanning

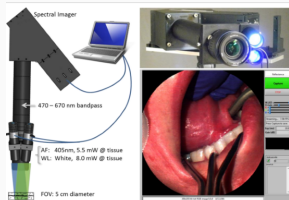


Snapshot

Spectral Imaging Applications



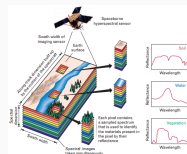
Art restoration ¹



Oral cancer diagnostics ²

Drawbacks

- Huge volumens of data
- It takes a long time to gather the data cube



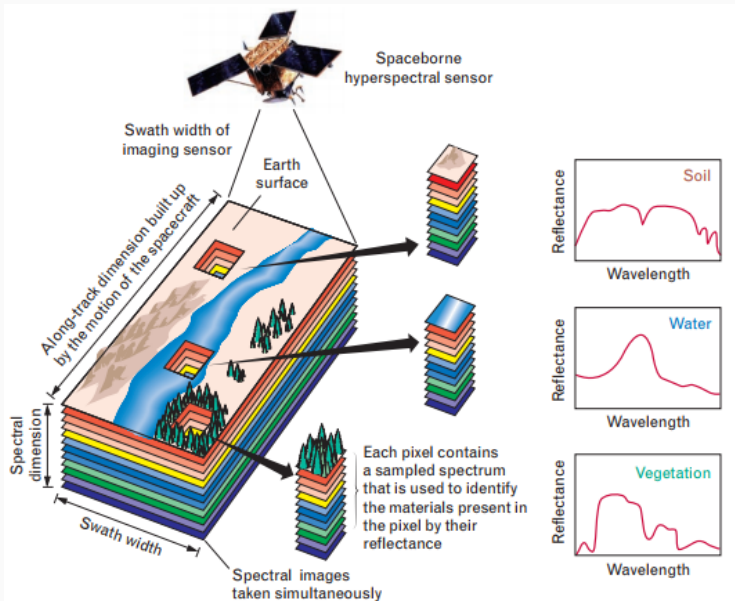
Classification ³

¹ <https://cen.acs.org/articles/93/i44/Medical-Imaging-Turns-Of-Neglected.html>

² Noah Bedard, 2013

³ Sellar, R. Glenn, 2005

What Classification Is?



Why to use Deep Learning (DL)?

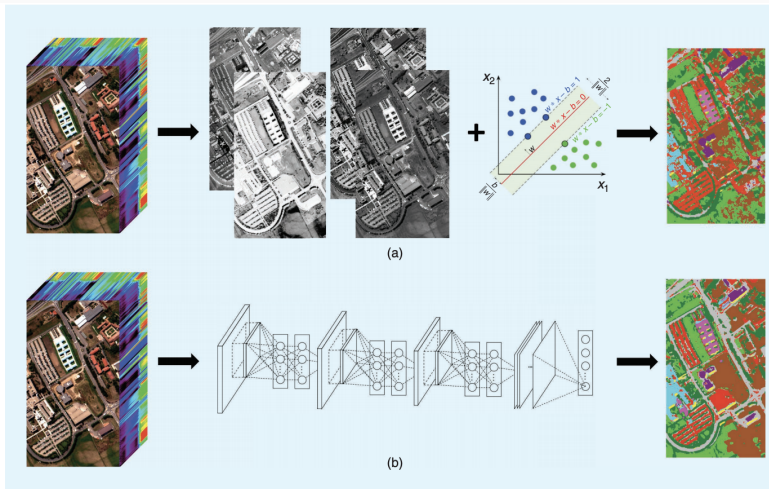
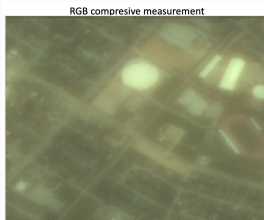
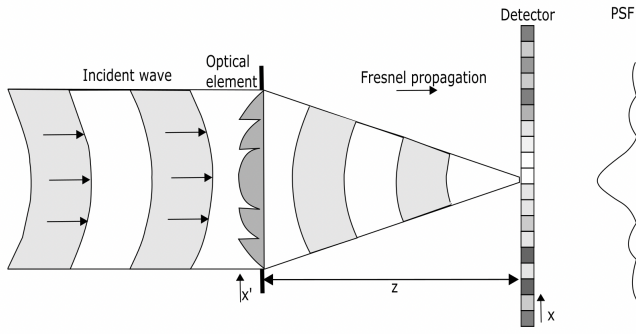


FIGURE 2. FE via (a) machine learning and (b) DL.

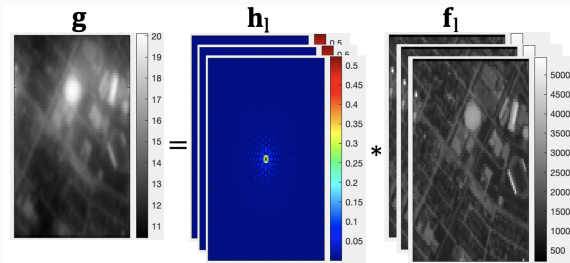
Diffractive Optics



Compressive Spectral Diffractive System

The compressive spectral diffractive measurement can be represented as:

$$\mathbf{g} = \text{Sensor}(\mathbf{h}_l * \mathbf{f}_l)_{\text{Bayer}} \quad (1)$$



where $\mathbf{g} \in \mathbf{R}^{M \times M}$. $\mathbf{f}_l \in \mathbf{R}^{M \times M}$ denotes the l^{th} an image in a spectral wavelength, $\mathbf{h}_l \in \mathbf{R}^{M \times M}$ denotes the point spread function (PSF), and $*$ is the convolution operation, $\omega_l \sim \mathcal{N}(0, \sigma^2)$

Point Spread Function and Lens Design

The pupil function is given by:

$$\mathcal{P}(x, y) = P(x, y) \frac{i2\pi\Delta}{\lambda} W(x, y) \quad (2)$$

The diffractive element introduces an phase delay in a complex-valued wave field which can be modeled as the optical phase

$\phi(x, y) = \frac{2\pi\Delta d}{\lambda} W(x, y)$, and $W(x, y)$ is the wave-front. The Point Spread Function (PSF) is computed as

$$\mathbf{h}_l = |\mathcal{F}\{\mathcal{P}(x, y)\}|^2 \quad (3)$$

the wave-front can be represented with a combination of Zernike polynomials:

$$\mathbf{W} = \sum_{n=0}^{N-1} \alpha_n \mathbf{Z}_n, \quad (4)$$

where N is the number of Zernike's modes, $\mathbf{Z}_n$ are the Zernike's polynomials, and α_n is the n^{th} coefficient.

Reconstruction: Wiener filtering

$$\min_{\tilde{\mathbf{f}}} \{ \|\mathbf{g} - \text{Sensor}(\mathbf{h}_l * \tilde{\mathbf{f}}_l)_{\text{Bayer}}\|_2^2 + \tau \|\tilde{\mathbf{f}}_l\|_2^2 \} \quad (5)$$

τ is a regularization parameter, $\tau = \frac{S_\omega}{S_f}$, S_ω is the power spectrum of the noise, and S_f is the power spectrum of the signal. First, it is obtained a demosaicked blur RGB image $\bar{\mathbf{f}}_l = \text{Demosaicking}(\mathbf{y}, \mathbf{B})$, once the demosaicking is performed, the Wiener filter might be applied

$$\hat{\mathbf{g}}_l = \mathcal{F}^{-1} \left\{ \frac{\mathbf{H}_l^*}{\mathbf{H}_l^2 + \tau} \bar{\mathbf{F}}_l \right\} \quad (6)$$

where $\bar{\mathbf{F}}_l$ correspond to the measurement in the Fourier domain, $\mathbf{H}_l^*$ is the complex conjugate of the optical transfer function $\mathbf{H}_l$, where $|\mathbf{H}_l|^2 = \mathbf{H}_l^* \mathbf{H}_l$ is the magnitude squared. The complex conjugate of complex number is $C^* = R - iI$.

Reconstruction: Richardson-Lucy algorithm

The Richardson-Lucy algorithm as the deconvolution algorithm, which uses an iterative approach based on back-projection of the discrepancy between the blurred color image $\bar{\mathbf{f}}_l$ and the reconstructed model

$$\hat{\mathbf{f}}_l^{k+1} = \hat{\mathbf{f}}_l^k \odot \left[\frac{\bar{\mathbf{f}}_l}{\mathbf{h}_l * \hat{\mathbf{f}}_l^k} * \mathbf{h}_l^\dagger \right], \quad (7)$$

where $\hat{\mathbf{f}}_l^k$ is the estimate of the original image at the k^{th} iteration, $*$ is the convolution operation, and $\mathbf{h}_l^\dagger$ is the mirrored PSF, i.e., the spatially flipped version of the PSF used to implement the transpose of the convolution operator during deconvolution, $\odot$ is the element-wise multiplication, the division is performed element-wise.

Zernike Circle Polynomials

According to the Noll notation the Zernikes circles polynomials are defined as

$$Z_n^m(r, \theta) = \sqrt{2(n+1)} R_n^m(r) G^m(\theta), \quad (8)$$

where m and n are non-negative integers, and $m \leq n$.

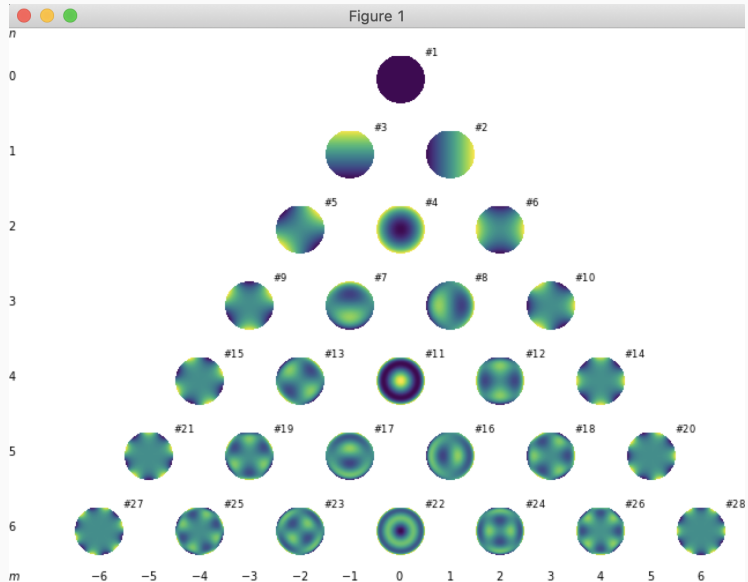
$$Z_i(r, \theta) = \{ \sqrt{2(n+1)} R_n^m(r) G^m(\theta) \} \quad (9)$$

The radial and azimuthal factors $R_n^m(r)$ and $G^m(\theta)$ are given by

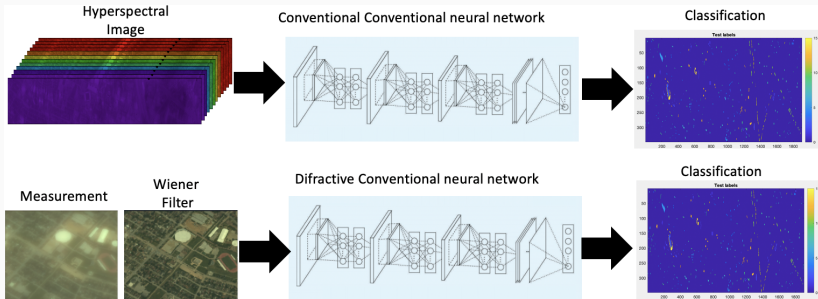
$$R_n^m(r) = \sum_{k=0}^{\frac{n-m}{2}} \frac{(-1)^k (n-k)!}{k! \left(\frac{n+m}{2} - k\right)! \left(\frac{n-m}{2} - k\right)!} r^{n-2k}$$
$$G^m(\theta) = \begin{cases} \sin(m\theta) & i \text{ odd} \\ \cos(m\theta) & i \text{ even} \end{cases}$$

$k! = \Gamma(k+1)$ executes faster than factorial function.

Zernike Polynomial Pyramid



Conventional-CNN Against Diffractive-CNN in Classification



Datasets: Houston Database ⁴, Pavia Database ⁵

#	Dataset	# pixels	# spectral bands	# classes
1	Houston	349×1095	144	15
2	Pavia	610×340	103	9

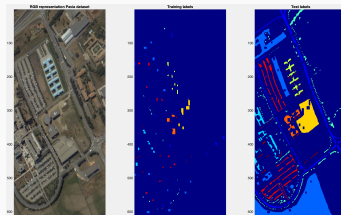
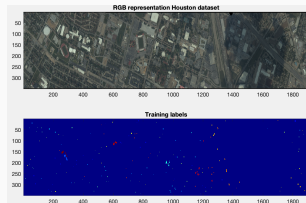
0 – Unclassified	9 – Road
1 – Healthy grass	10 – Highway
2 – Stressed grass	11 – Railway
3 – Synthetic grass	12 – Parking Lot 1
4 – Trees	13 – Parking Lot 2
5 – Soil	14 – Tennis Court
6 – Water	15 – Running Track
7 – Residential	
8 – Commercial	

0 – Unclassified	5 – Bitumen
1 – Water	6 – Tiles
2 – Trees	7 – Shadows
3 – Asphalt	8 – Meadows
4 – Self-Blocking Bricks	9 – Bare Soil

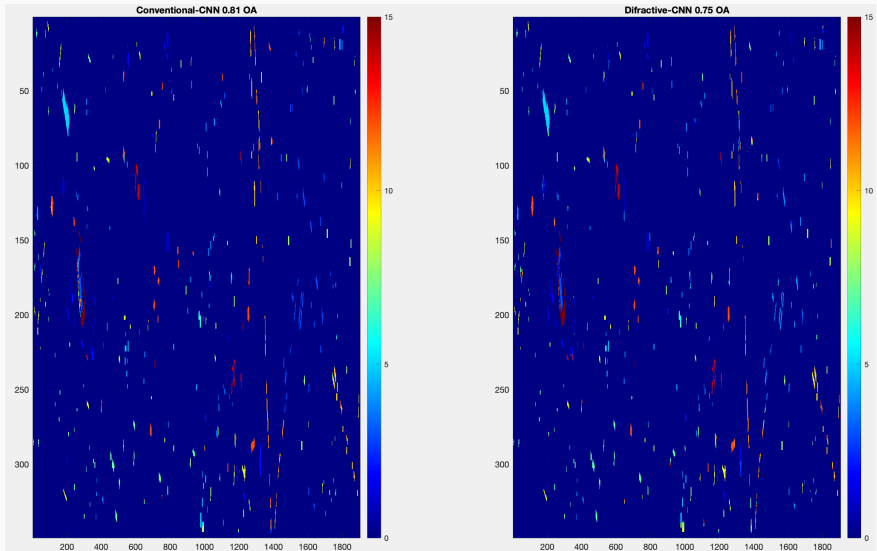
Software setup: Matlab 2019 (UIS), Python and Tensorflow. **Hardware setup:** CPU: Intel(R) i7-1075 2.6GHz, GPU: NVIDIA RTX 2060, RAM: 32 GB (PUCV).

⁴https://hyperspectral.ee.uh.edu/?page_id=459

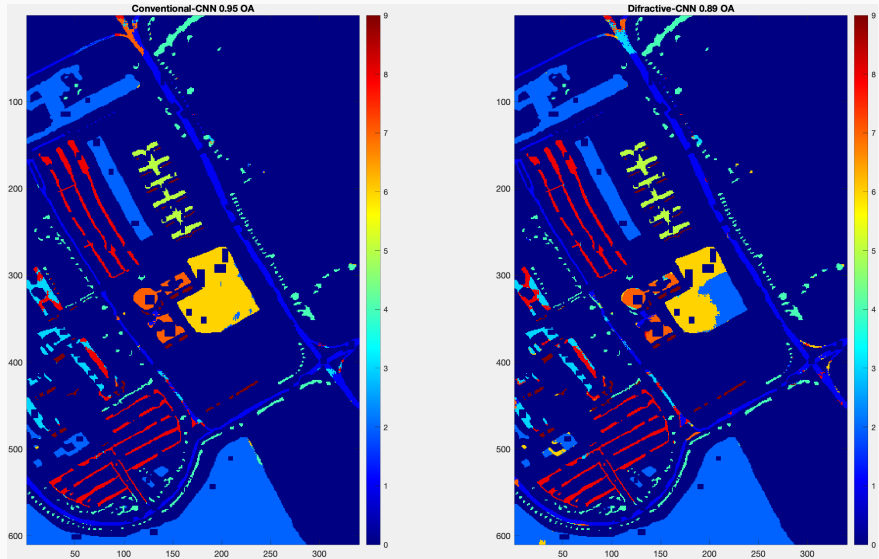
⁵http://www.ehu.es/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes#Indian_Pines



Simulations Results Houston data set



Simulations Results Pavia data set



Classification comparison per class, Houston data set

#	Classes	Conventional-CNN	Diffraction-CNN
1	Healthy grass	0.8148	0.6885
2	Stressed grass	0.8251	0.6522
3	Synthetic grass	0.7643	0.6831
4	Trees	0.9195	0.9024
5	Soil	0.9535	0.8683
6	Water	0.7902	0.7762
7	Residential	0.8600	0.8152
8	Commercial	0.6885	0.6163
9	Road	0.8734	0.8177
10	Highway	0.5308	0.5212
11	Railway	0.7390	0.6470
12	Parking Lot 1	0.9135	0.8799
13	Parking Lot 2	0.9017	0.8596
14	Tennis Court	1.0000	0.9838
15	Running Track	0.7885	0.9069

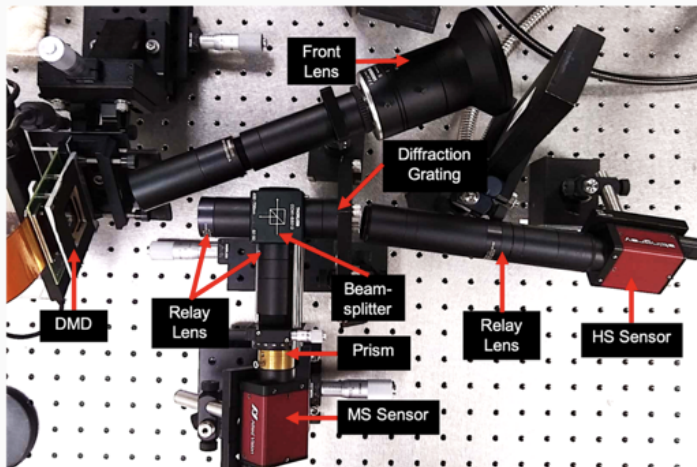
Table 1: Classification comparison using the Houston data set 2013 the CNN-state-of-the-art, and the proposed method.

Classification comparison per class, Pavia data set

#	Class	Samples	Conventional-CNN	Diffractional-CNN
1	Water	824	0.8862	0.8350
2	Trees	820	0.9964	0.9914
3	Asphalt	816	0.7068	0.5944
4	Self-Blocking Bricks	808	0.9395	0.8990
5	Bitumen	808	0.9946	0.9955
6	Tiles	1260	0.9774	0.6535
7	Shadows	476	0.9378	0.7900
8	Meadows	824	0.9892	0.9836
9	Bare Soil	820	0.9773	0.9283

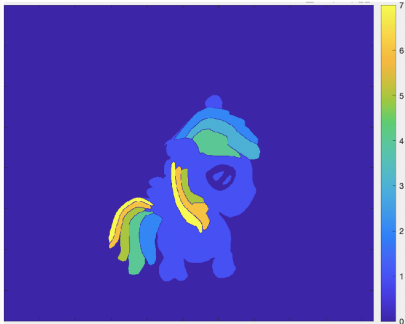
Table 2: Classification comparison using Pavia University data set between the CNN-state-of-the-art, and the proposed method.

Experiments with Real-data

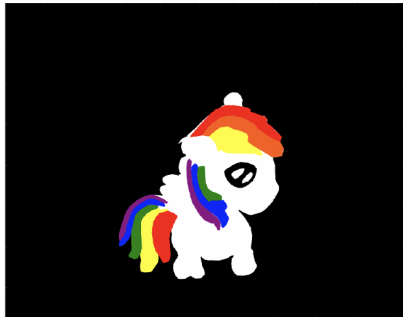


Real-Data Experiments

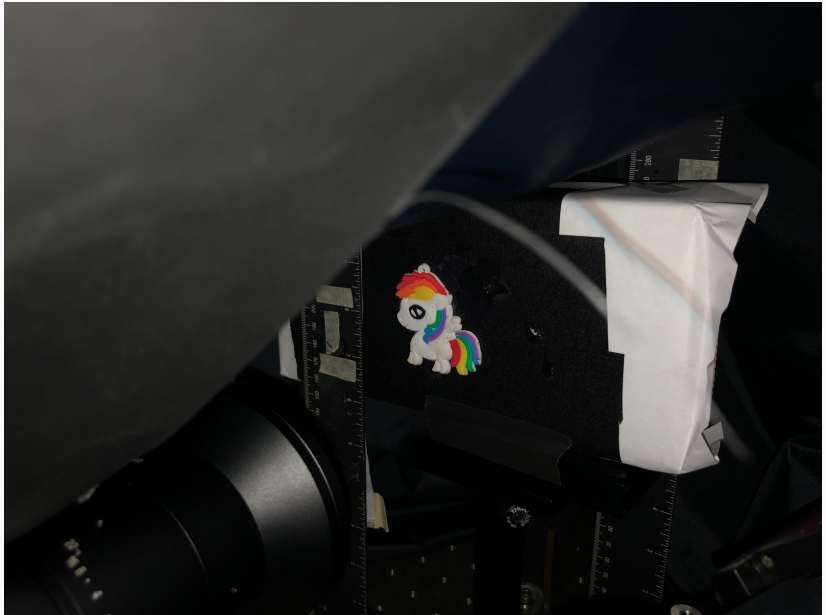
Groundtruth of the scene



RGB version of the scene



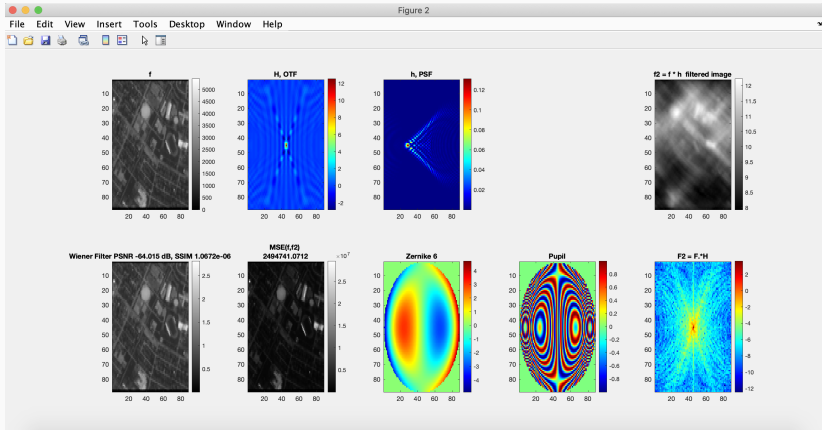
Real-Data Experiments



To design the lens using Zernike polynomials

To capture diffractive measurements.

To classify based on the diffractive measurement.



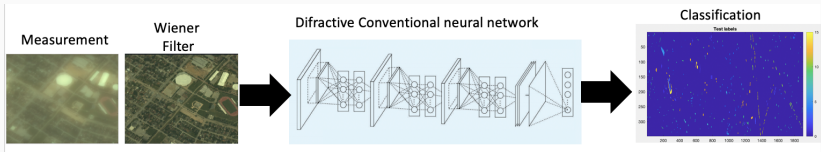
Conclusions

It is presented the diffractive sensing model.

It is compared the conventional-CNN (0.81 OA) against the diffractive-CNN (0.75 OA).

It is presented an alternative scenario for the testing with real data.

It is shown the software to the design the parameters of the sensing system.



Questions?

